Лабораторна робота №3. Розгортання власного застосунку в Kubernetes

Дисципліна: «Технології DevOps» · Студент: MarcoGr

Репозиторій: https://github.com/MarcoGr11/DevOps

У namespace `coworking-booking` розгорнуто систему бронювання коворкінгу: api (Django REST, 2 репліки), worker (2 репліки) і PostgreSQL з томом PVC, налаштування в ConfigMap, паролі в Secret, доступ ззовні через Ingress `booking.local`. Середовище: minikube, Kubernetes v1.37.0 (kubectl v1.37.1). Образи: `ghcr.io/marcogr11/devops-api:1.0.0` і `:1.1.0` (у 1.1.0 змінюється поле `version` у відповіді `/api/health/`), worker — `ghcr.io/marcogr11/devops-worker:1.0.0`.

5. Розгортання

5.a Secret і apply

Створюємо Secret з випадковими паролями і застосовуємо всі маніфести з `k8s/`.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl create secret generic booking-secret -n coworking-booking \
  --from-literal=POSTGRES_PASSWORD="$(openssl rand -hex 16)" \
  --from-literal=DJANGO_SECRET_KEY="$(openssl rand -hex 32)"
kubectl apply -f k8s/ -n coworking-booking
secret/booking-secret created
deployment.apps/api created
ingress.networking.k8s.io/api created
service/api created
configmap/booking-config created
Warning: resource namespaces/coworking-booking is missing the kubectl.kubernetes.io/last-applied-configuration annotation which is required by kubectl apply. kubectl apply should only be used on resource
s created declaratively by either kubectl create --save-config or kubectl apply. The missing annotation will be patched automatically.
namespace/coworking-booking configured
deployment.apps/postgres created
persistentvolumeclaim/postgres-data created
service/postgres created
deployment.apps/worker created
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl annotate deployment/api -n coworking-booking kubernetes.io/change-cause="initial deploy: image 1.0.0"

deployment.apps/api annotated
```

Рис. 1. Створення Secret і застосування маніфестів

`secret/booking-secret created`, далі `created` для Deployment-ів api, worker і postgres, обох Service-ів, Ingress, ConfigMap і PVC `postgres-data`. Namespace перед цим створено командою `kubectl create namespace`, тому `apply` вивів `Warning` про відсутню анотацію `last-applied-configuration`, сам її додав (`namespace/coworking-booking configured`). Значення паролів у терміналі не з'являються.

5.b Об'єкти в namespace

Перевіряємо, що всі поди запустились і об'єкти створені.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get pods -n coworking-booking -o wide

NAME                        READY   STATUS    RESTARTS      AGE   IP           NODE       NOMINATED NODE   READINESS GATES
api-6469f9dd88-m5rp5        1/1     Running   2 (76s ago)   80s   10.244.0.5   minikube   <none>           <none>
api-6469f9dd88-zjf7h        1/1     Running   2 (76s ago)   80s   10.244.0.6   minikube   <none>           <none>
postgres-747ccbb877-tvrpd   1/1     Running   0             79s   10.244.0.7   minikube   <none>           <none>
worker-6cf4b4d54d-grrln     1/1     Running   2 (76s ago)   79s   10.244.0.8   minikube   <none>           <none>
worker-6cf4b4d54d-z88th     1/1     Running   2 (76s ago)   79s   10.244.0.9   minikube   <none>           <none>
( .venv) admin@MacBook-Pro-admin DevOps lab1 %
 (.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get deployments,replicasets,services,ingress,pvc,configmap,secret -n coworking-booking

 NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
 deployment.apps/api          2/2     2            2           103s
 deployment.apps/postgres     1/1     1            1           102s
 deployment.apps/worker       2/2     2            2           102s

 NAME                                    DESIRED   CURRENT   READY   AGE
 replicaset.apps/api-6469f9dd88          2         2         2       103s
 replicaset.apps/postgres-747ccbb877     1         1         1       102s
 replicaset.apps/worker-6cf4b4d54d       2         2         2       102s

 NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
 service/api          ClusterIP   10.96.96.26      <none>        80/TCP     103s
 service/postgres     ClusterIP   10.107.58.134    <none>        5432/TCP   102s

 NAME                              CLASS   HOSTS           ADDRESS        PORTS   AGE
 ingress.networking.k8s.io/api     nginx   booking.local   192.168.49.2   80      103s

 NAME                                     STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
 persistentvolumeclaim/postgres-data      Bound    pvc-ad77561e-ac4f-42ac-b705-2e62ef25b550   1Gi        RWO            standard       <unset>                 102s

 NAME                         DATA   AGE
 configmap/booking-config     8      103s
 configmap/kube-root-ca.crt   1      5m59s

 NAME                     TYPE     DATA   AGE
 secret/booking-secret    Opaque   2      3m12s
```

Рис. 2. Поди, Deployment-и, ReplicaSet-и, Service-и, Ingress, PVC після розгортання

Усі 5 подів у стані `Running` і `1/1`: `api-6469f9dd88-m5rp5`, `api-6469f9dd88-zjf7h`, два worker і `postgres-747ccbb877-tvrpd`. Deployment-и `api 2/2`, `worker 2/2`, `postgres 1/1`, PVC `postgres-data` у стані `Bound` (1Gi), Ingress на `booking.local`. `RESTARTS 2` у api та worker — це перезапуски на старті, поки postgres ще не приймав з'єднань.

5.c /api/health/ у браузері

Відкриваємо health-ендпоінт застосунку в браузері.

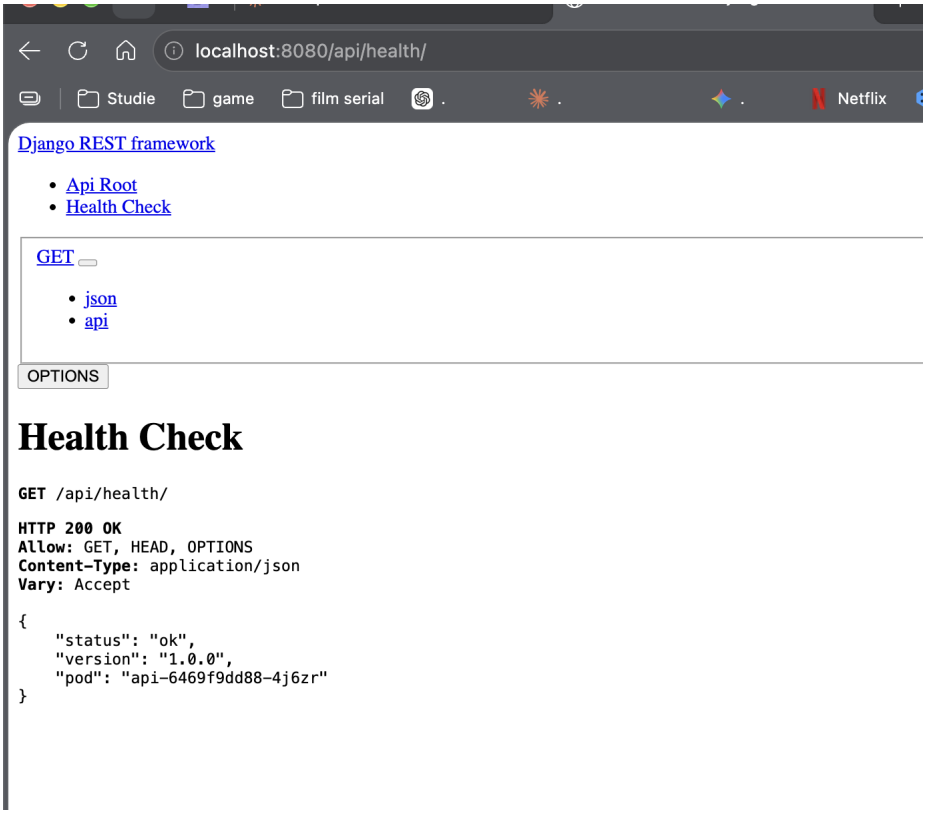


Рис. 3. Відповідь /api/health/ у браузері

В адресному рядку `localhost:8080/api/health/` (port-forward на Service api). Django REST framework повертає `HTTP 200 OK` з JSON `"status": "ok"`, `"version": "1.0.0"`, а поле `pod` показує под, який обробив запит: `api-6469f9dd88-4j6zr`.

5.d ConfigMap і Secret у контейнері

Перевіряємо, що змінні з ConfigMap і Secret потрапили в контейнер api, не показуючи паролів.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl exec -n coworking-booking deploy/api -- env | grep -E "POSTGRES_(USER|DB|HOST)|DJANGO_DEBUG|BOOKING_" | sort
for k in POSTGRES_PASSWORD DJANGO_SECRET_KEY; do
  echo "$k  Secret: $(kubectl get secret booking-secret -n coworking-booking -o jsonpath="{.data.$k}" | base64 -d | shasum -a 256 | cut -c1-12)  pod: $(kubectl exec -n coworking-booking deploy/api -- pri
ntenv $k | tr -d '\n' | shasum -a 256 | cut -c1-12)"
done
BOOKING_TIMEOUT_CHECK_INTERVAL=30
BOOKING_TIMEOUT_MINUTES=15
DJANGO_DEBUG=0
POSTGRES_DB=booking
POSTGRES_HOST=postgres
POSTGRES_USER=booking
POSTGRES_PASSWORD  Secret: be3d4efaec0d  pod: be3d4efaec0d
DJANGO_SECRET_KEY  Secret: 1347ce6b64f2  pod: 1347ce6b64f2
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 4. Змінні ConfigMap і хеші значень Secret у контейнері api

У контейнері api є всі значення з ConfigMap: `POSTGRES_DB=booking`, `POSTGRES_HOST=postgres`, `POSTGRES_USER=booking`, `DJANGO_DEBUG=0`, `BOOKING_TIMEOUT_CHECK_INTERVAL=30`, `BOOKING_TIMEOUT_MINUTES=15`. Для кожного ключа Secret хеш значення в Secret і в поді однаковий (`be3d4efaec0d` для пароля БД, `1347ce6b64f2` для ключа Django): паролі потрапили в контейнер, хоча самих значень на екрані немає.

6. Життєвий цикл

6.a Самовідновлення

Видаляємо под api вручну і дивимось, як Kubernetes створює заміну.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get pods -n coworking-booking -l app=api

NAME                   READY   STATUS    RESTARTS        AGE
api-6469f9dd88-m5rp5   1/1     Running   2 (4m42s ago)   4m46s
api-6469f9dd88-zjf7h   1/1     Running   2 (4m42s ago)   4m46s
(.venv) admin@MacBook-Pro-admin DevOps lab1 %

(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl delete pod -n coworking-booking $(kubectl get pods -n coworking-booking -l app=api -o jsonpath='{.items[0].metadata.name}')

pod "api-6469f9dd88-m5rp5" deleted from coworking-booking namespace

(.venv) admin@MacBook-Pro-admin DevOps lab1 %
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get pods -n coworking-booking -l app=api

NAME                   READY   STATUS    RESTARTS        AGE
api-6469f9dd88-vq8km   1/1     Running   0               53s
api-6469f9dd88-zjf7h   1/1     Running   2 (5m53s ago)   5m57s
```

Рис. 5. Видалення пода api і його автоматична заміна

Видалено под `api-6469f9dd88-m5rp5`. Після цього знову працюють 2 поди: старий `api-6469f9dd88-zjf7h` (AGE 5m57s) і новий `api-6469f9dd88-vq8km` (AGE 53s, RESTARTS 0), якого ReplicaSet створив замість видаленого.

6.b Масштабування до 4

Збільшуємо кількість реплік api до 4 і перевіряємо, що запити розходяться між усіма подами.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl scale deployment api -n coworking-booking --replicas=4

deployment.apps/api scaled
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl rollout status deployment/api -n coworking-booking

deployment "api" successfully rolled out
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get pods -n coworking-booking -l app=api -o wide

NAME                   READY   STATUS    RESTARTS        AGE    IP            NODE       NOMINATED NODE   READINESS GATES
api-6469f9dd88-c4wrz   1/1     Running   0               32s    10.244.0.11   minikube   <none>           <none>
api-6469f9dd88-l5bzc   1/1     Running   0               32s    10.244.0.12   minikube   <none>           <none>
api-6469f9dd88-vq8km   1/1     Running   0               119s   10.244.0.10   minikube   <none>           <none>
api-6469f9dd88-zjf7h   1/1     Running   2 (6m59s ago)   7m3s   10.244.0.6    minikube   <none>           <none>
(.venv) admin@MacBook-Pro-admin DevOps lab1 % for i in $(seq 1 40); do curl -s -H "Host: booking.local" http://localhost:8081/api/health/ | grep -o '"pod":"[^"]*"'; done | sort | uniq -c

 10 "pod":"api-6469f9dd88-c4wrz"
  9 "pod":"api-6469f9dd88-l5bzc"
 10 "pod":"api-6469f9dd88-vq8km"
 11 "pod":"api-6469f9dd88-zjf7h"
```

Рис. 6. Масштабування api до 4 реплік і розподіл запитів

Працюють 4 поди api (`c4wrz`, `l5bzc`, `vq8km`, `zjf7h`) з різними IP 10.244.0.6–0.12. 40 запитів через Ingress розподілились між усіма чотирма приблизно порівну: 10, 9, 10 і 11.

6.c Rolling update на 1.1.0

Оновлюємо образ api з 1.0.0 на 1.1.0. Перед цим у другому вікні запускаємо цикл curl (6.d).

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl set image deployment/api api=ghcr.io/marcogr11/devops-api:1.1.0 -n coworking-booking

deployment.apps/api image updated
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl annotate deployment/api -n coworking-booking kubernetes.io/change-cause="update image to 1.1.0" --overwrite

deployment.apps/api annotated
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl rollout status deployment/api -n coworking-booking

deployment "api" successfully rolled out
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get pods,rs -n coworking-booking -l app=api

NAME                         READY   STATUS        RESTARTS        AGE
pod/api-6469f9dd88-zjf7h     1/1     Terminating   2 (9m42s ago)   9m46s
pod/api-84b8bccfdf-jt25d     1/1     Running       0               44s
pod/api-84b8bccfdf-km8s6     1/1     Running       0               50s
pod/api-84b8bccfdf-qbk48     1/1     Running       0               38s
pod/api-84b8bccfdf-qh8td     1/1     Running       0               32s

NAME                                  DESIRED   CURRENT   READY   AGE
replicaset.apps/api-6469f9dd88        0         0         0       9m46s
replicaset.apps/api-84b8bccfdf        4         4         4       50s
```

Рис. 7. Rolling update api з 1.0.0 на 1.1.0

Після `set image` старий ReplicaSet `api-6469f9dd88` зменшився до 0, а новий `api-84b8bccfdf` виріс до 4 (усі `Ready`). Останній старий под `api-6469f9dd88-zjf7h` ще у стані `Terminating`, а чотири нові `api-84b8bccfdf-*` уже `Running` з AGE 32–50 с: їх запускали по одному.

6.d Доступність під час оновлення

Перевіряємо, що під час оновлення сервіс відповідав без помилок.



Рис. 8. Цикл запитів під час rolling update і підсумок

Під час оновлення відправлено 708 запитів, і всі 708 повернули HTTP 200, жодної помилки. Спершу відповідали поди 1.0.0 (330 відповідей), потім 1.1.0 (378), тобто версія змінилась без простою.

6.e Історія і відкат

Дивимось історію ревізій, відкочуємось на 1.0.0 і перевіряємо версію.

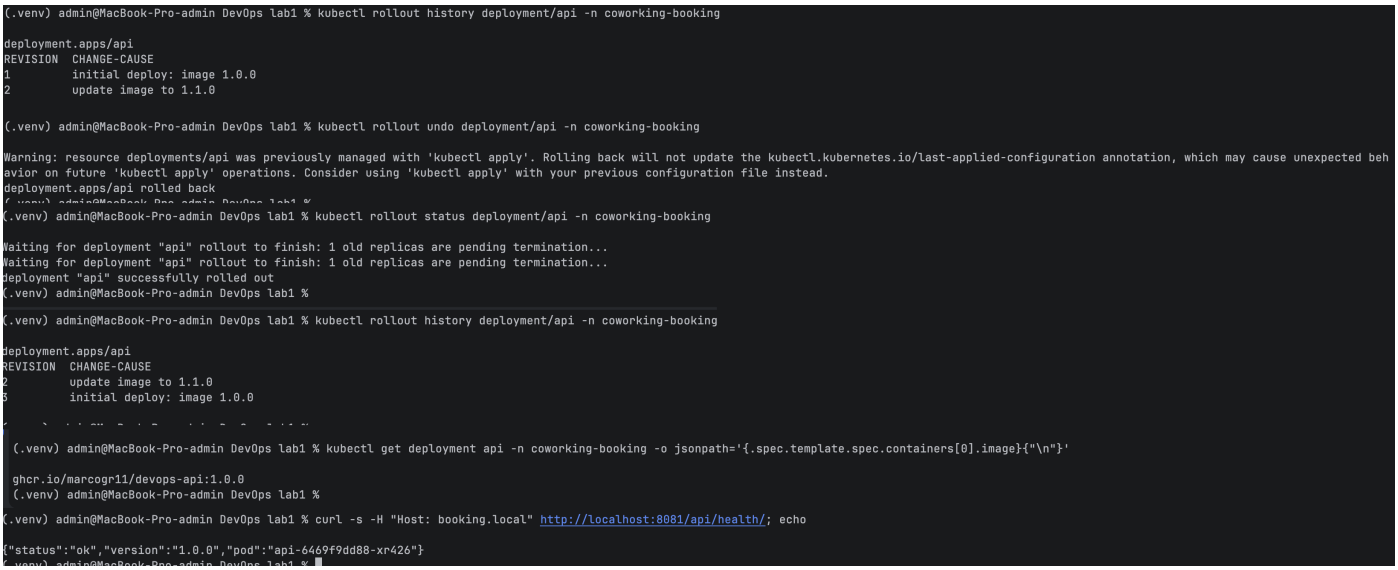


Рис. 9. Історія ревізій і відкат на 1.0.0

До відкату в історії ревізії 1 (`initial deploy: image 1.0.0`) і 2 (`update image to 1.1.0`). Після `rollout undo` ревізія 1 стала новою ревізією 3, образ знову `devops-api:1.0.0`, і `/api/health/` відповідає `"version":"1.0.0"`. `Warning` про `kubectl apply` — стандартне попередження: undo не оновлює анотацію last-applied-configuration.

7. Діагностика

7.a Неіснуючий тег 9.9.9

Вказуємо в маніфесті неіснуючий тег образу і застосовуємо його.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % sed -i '' 's#devops-api:1.0.0#devops-api:9.9.9#' k8s/api-deployment.yaml

(.venv) admin@MacBook-Pro-admin DevOps lab1 %

(.venv) admin@MacBook-Pro-admin DevOps lab1 % grep -n "image:" k8s/api-deployment.yaml

24:          image: ghcr.io/marcogr11/devops-api:9.9.9
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl apply -f k8s/api-deployment.yaml -n coworking-booking

deployment.apps/api configured
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get pods -n coworking-booking -l app=api

NAME                   READY   STATUS         RESTARTS   AGE
api-6469f9dd88-qdts6   1/1     Running        0          4m5s
api-6469f9dd88-xr426   1/1     Running        0          3m53s
api-64d5fbbf6d-rl772   0/1     ErrImagePull   0          57s
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl rollout status deployment/api -n coworking-booking --timeout=5s

Waiting for deployment "api" rollout to finish: 1 out of 2 new replicas have been updated...
error: timed out waiting for the condition
(.venv) admin@MacBook-Pro-admin DevOps lab1 % curl -s -H "Host: booking.local" http://localhost:8081/api/health/; echo

{"status":"ok","version":"1.0.0","pod":"api-6469f9dd88-qdts6"}
```

Рис. 10. Оновлення на неіснуючий тег 9.9.9

Новий под `api-64d5fbbf6d-rl772` у стані `ErrImagePull` (`0/1`), а два старі поди `api-6469f9dd88-*` далі `Running`. `rollout status` завершився за таймаутом: оновлено 1 з 2 реплік. Застосунок при цьому доступний: curl отримав відповідь `"status":"ok"` від старого пода `api-6469f9dd88-qdts6`.

7.b describe і events

Шукаємо причину через опис пода і події.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % BAD=$(kubectl get pods -n coworking-booking -l app=api --field-selector=status.phase=Pending -o jsonpath='{.items[0].metadata.name}')

(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl describe pod $BAD -n coworking-booking | tail -12

Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason     Age                From               Message
  ----     ------     ----               ----               -------
  Normal   Scheduled  2m39s              default-scheduler  Successfully assigned coworking-booking/api-64d5fbbf6d-rl772 to minikube
  Normal   Pulling    70s (x4 over 2m39s)  kubelet          spec.containers{api}: Pulling image "ghcr.io/marcogr11/devops-api:9.9.9"
  Warning  Failed     70s (x4 over 2m38s)  kubelet          spec.containers{api}: Failed to pull image "ghcr.io/marcogr11/devops-api:9.9.9": rpc error: code = NotFound desc = failed to pull and unpack
image "ghcr.io/marcogr11/devops-api:9.9.9": failed to resolve reference "ghcr.io/marcogr11/devops-api:9.9.9": ghcr.io/marcogr11/devops-api:9.9.9: not found
  Warning  Failed     70s (x4 over 2m38s)  kubelet          spec.containers{api}: Error: ErrImagePull
  Normal   BackOff    4s (x10 over 2m37s)  kubelet          spec.containers{api}: Back-off pulling image "ghcr.io/marcogr11/devops-api:9.9.9"
  Warning  Failed     4s (x10 over 2m37s)  kubelet          spec.containers{api}: Error: ImagePullBackOff
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl get events -n coworking-booking --field-selector involvedObject.name=$BAD --sort-by=.lastTimestamp

LAST SEEN   TYPE      REASON      OBJECT                     MESSAGE
2m57s       Normal    Scheduled   pod/api-64d5fbbf6d-rl772   Successfully assigned coworking-booking/api-64d5fbbf6d-rl772 to minikube
88s         Normal    Pulling     pod/api-64d5fbbf6d-rl772   Pulling image "ghcr.io/marcogr11/devops-api:9.9.9"
88s         Warning   Failed      pod/api-64d5fbbf6d-rl772   Failed to pull image "ghcr.io/marcogr11/devops-api:9.9.9": rpc error: code = NotFound desc = failed to pull and unpack image "ghcr.io/marcogr1
1/devops-api:9.9.9": failed to resolve reference "ghcr.io/marcogr11/devops-api:9.9.9": ghcr.io/marcogr11/devops-api:9.9.9: not found
88s         Warning   Failed      pod/api-64d5fbbf6d-rl772   Error: ErrImagePull
10s         Normal    BackOff     pod/api-64d5fbbf6d-rl772   Back-off pulling image "ghcr.io/marcogr11/devops-api:9.9.9"
10s         Warning   Failed      pod/api-64d5fbbf6d-rl772   Error: ImagePullBackOff
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
```

Рис. 11. Події пода з помилкою завантаження образу

В Events пода `api-64d5fbbf6d-rl772` kubelet 4 рази пробував завантажити `ghcr.io/marcogr11/devops-api:9.9.9` і отримав `not found`, після чого перейшов у `Back-off pulling image` / `ImagePullBackOff`. Отже, причина в тому, що образу з таким тегом немає в реєстрі.

7.c Логи

Дивимось логи зламаного пода і робочих api та worker.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl logs $BAD -n coworking-booking

Error from server (BadRequest): container "api" in pod "api-64d5fbbf6d-rl772" is waiting to start: image can't be pulled
(.venv) admin@MacBook-Pro-admin DevOps lab1 %
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl logs deploy/api -n coworking-booking --tail=8

Found 3 pods, using pod/api-6469f9dd88-qdts6
Operations to perform:
  Apply all migrations: admin, auth, booking, contenttypes, sessions
Running migrations:
  No migrations to apply.
[2026-10-19 22:06:37 +0000] [8] [INFO] Starting gunicorn 22.0.0
[2026-10-19 22:06:37 +0000] [8] [INFO] Listening at: http://0.0.0.0:8000 (8)
[2026-10-19 22:06:37 +0000] [8] [INFO] Using worker: sync
[2026-10-19 22:06:37 +0000] [9] [INFO] Booting worker with pid: 9
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl logs deploy/worker -n coworking-booking --tail=5

Found 2 pods, using pod/worker-6cf4b4d54d-grrln
    cur.execute(
psycopg2.errors.UndefinedTable: relation "booking_booking" does not exist
LINE 2:             UPDATE booking_booking
                           ^
```

Рис. 12. Логи зламаного пода, api і worker

Зламаний под логів не має, бо контейнер не стартував (`waiting to start: image can't be pulled`), тому для ImagePullBackOff дивляться events. Робочий под api застосував міграції (`No migrations to apply`) і запустив gunicorn на `0.0.0.0:8000`. Останні рядки worker — помилка `relation "booking_booking" does not exist` зі старту: worker запустився раніше, ніж api створив таблиці. Після цього под не перезапускався (RESTARTS 2 лише на старті, див. 7.e).

7.d Вхід у контейнер

Заходимо в робочий контейнер api і перевіряємо його зсередини.

```
(.venv) admin@MacBook-Pro-admin DevOps lab1 % kubectl exec -it deploy/api -n coworking-booking -- sh

# whoami; hostname
root
api-6469f9dd88-qdts6
# ls /app
Dockerfile  booking  config  manage.py  requirements.txt  staticfiles
# env | grep -E "POSTGRES_(USER|DB|HOST)|DJANGO_DEBUG"
POSTGRES_HOST=postgres
POSTGRES_USER=booking
DJANGO_DEBUG=0
POSTGRES_DB=booking
# python -c 'import urllib.request;print(urllib.request.urlopen("http://localhost:8000/api/health/").read().decode())'
{"status":"ok","version":"1.0.0","pod":"api-6469f9dd88-qdts6"}
# python manage.py showmigrations booking
booking
 [X] 0001_initial
 [X] 0002_alter_resource_options
#
```

Рис. 13. Інтерактивний shell у контейнері api

Shell відкрився в поді `api-6469f9dd88-qdts6` від імені `root`. У `/app` лежить код Django-проєкту, змінні з ConfigMap на місці (`POSTGRES_HOST=postgres`, `DJANGO_DEBUG=0`). Застосунок відповідає зсередини на `localhost:8000`, обидві міграції `booking` застосовані (`[X]`).

7.e Виправлення

Повертаємо тег 1.0.0 у маніфест і застосовуємо знову.

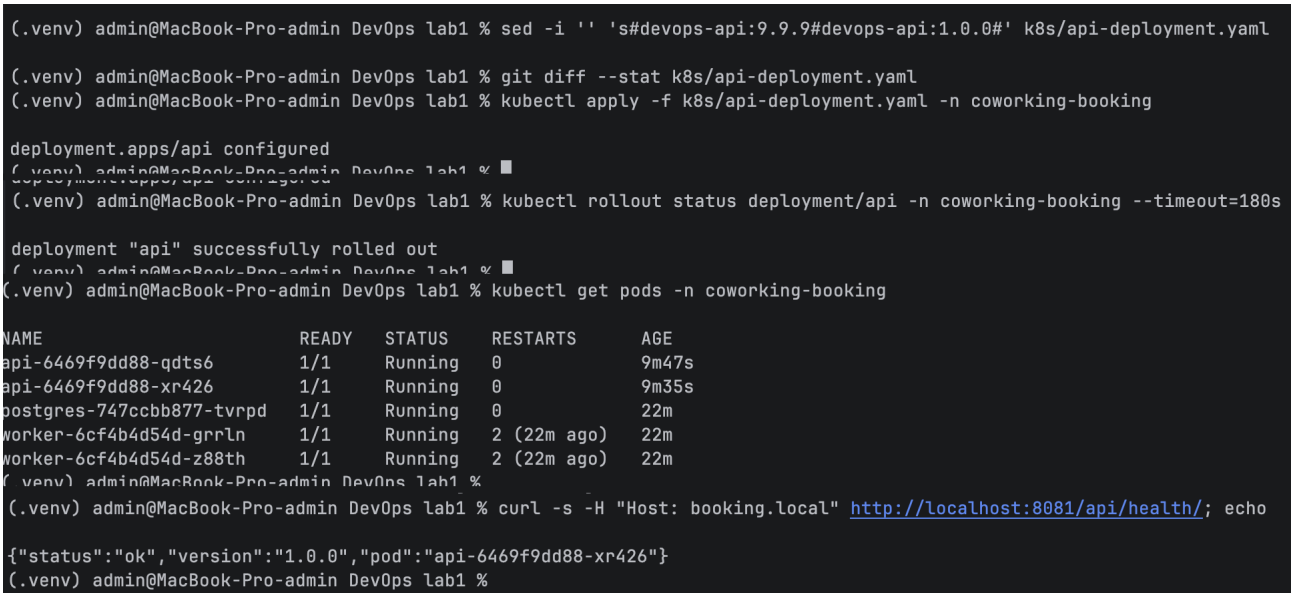


Рис. 14. Повернення тегу 1.0.0 і відновлення rollout

`git diff` порожній, тобто маніфест знову такий, як у git. Rollout завершився (`successfully rolled out`), под з помилкою прибрано, і всі 5 подів `Running`. `/api/health/` відповідає `"status":"ok"`, версія 1.0.0.

Висновок

Застосунок розгорнуто в minikube з маніфестів, а значення з ConfigMap і Secret потрапили в контейнер api (хеші паролів у Secret і в поді збігаються). Показано самовідновлення після видалення пода, масштабування до 4 реплік із розподілом запитів, rolling update на 1.1.0 без жодної помилки серед 708 запитів і відкат на 1.0.0. На прикладі неіснуючого тегу 9.9.9 розібрано діагностику через get pods, describe, events, logs і exec: старі поди продовжували обслуговувати запити, поки помилку не виправили.